FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	CRUZ, KRISTA
Section:	AN41
Professor:	MS. TAN
Date:	10/03/2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

Your Code - Laboratory Tasks

```
facts = {
    "temperature": 34,
    "is_raining": True,
    "humidity": 90,
    "smoke_detected": False,
    "occupancy": True
}

def rule_hot(kb):
    if kb.get("temperature", 0) > 30:
        return "Turn on the aircon"

def rule_rain(kb):
    if kb.get("is_raining", False):
        return "Bring umbrella"

def rule_smoke(kb):
    if kb.get("smoke_detected", False):
        return "Trigger fire alarm"

rules = [rule_hot, rule_rain, rule_smoke]

def run_inference_engine(kb, rule_base):
    inferred_actions = []
    for rule in rule_base:
        action = rule(kb)
        if action:
            inferred_actions.append(action)
    return inferred_actions

case_base = [
    {
        "problem": {"won_t_turn_on": True, "strange_noise": False, "indicator_light_off": True},
        "solution": "Check the power cable and outlet connection."
    },
    {
        "problem": {"won_t_turn_on": False, "strange_noise": True, "indicator_light_off": False},
        "solution": "Clear obstructions from the motor and check bearings."
    },
    {
        "problem": {"won_t_turn_on": True, "strange_noise": True, "indicator_light_off": True},
        "solution": "Perform a hard reset and inspect the internal fuse."
```

```
    }
]

def retrieve_and_reuse(new_problem, cases):
    print("\n[RETRIEVE & REUSE]")
    best_match_solution = None
    max_similarity = -1

    for case in cases:
        # calculate overlap similarity matching based on matching feature values
        past_problem = case["problem"]
        match_count = sum(1 for k, v in new_problem.items() if past_problem.get(k) == v)

        if match_count > max_similarity:
            max_similarity = match_count
            best_match_solution = case["solution"]

    print("Matching problem found in Case Base.")
    print(f"Retrieved Solution: {best_match_solution}")
    return best_match_solution

def revise_solution(solution):
    # allow adjustment or return unchanged
    print(f"\n[REVISE] Retrieved Solution: {solution}")
    choice = input("Do you want to adjust this solution? (y/n): ").strip().lower()
    if choice == 'y':
        return input("Enter revised solution: ").strip()
    return solution

def retain_case(new_problem, final_solution, cases):
    new_case = {"problem": new_problem, "solution": final_solution}
    cases.append(new_case)
    print("[RETAIN] New case successfully added to the case base.")

if __name__ == "__main__":
    print("=== Rule-Based Reasoning (RBR) Output ===")
    actions = run_inference_engine(facts, rules)
    print("Rule-Based Reasoning Actions:")
    for action in actions:
        print(f"- {action}")

    print("\n=== Case-Based Reasoning (CBR) Output ===")
    current_problem = {"won_t_turn_on": True, "strange_noise": False, "indicator_light_off": True}
```

```
retrieved_solution = retrieve_and_reuse(current_problem, case_base)

final_solution = revise_solution(retrieved_solution)
retain_case(current_problem, final_solution, case_base)

print(f"\nTotal cases in Case Base now: {len(case_base)}")
```

Output:

```
=== Rule-Based Reasoning (RBR) Output ===
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella

=== Case-Based Reasoning (CBR) Output ===

[RETRIEVE & REUSE]
Matching problem found in Case Base.
Retrieved Solution: Check the power cable and outlet connection.

[REVISE] Retrieved Solution: Check the power cable and outlet connection.
Do you want to adjust this solution? (y/n): n
[RETAIN] New case successfully added to the case base.

Total cases in Case Base now: 4
```

Comparative Reflection:

1. Which system (RBR or CBR) was easier to implement? Why?
    - RBR is easier because it uses simple IF-THEN rules. If a condition is met, it gives a direct answer. CBR requires extra logic to calculate similarity scores and match problems to past cases.
2. Which is more flexible in solving unstructured problems?
    - CBR is more flexible because it relies on past cases rather than rules. It can handle unexpected situations by finding the closest match.
3. Which is better for situations with continuously changing data?
    - CBR is better because whenever it solves a new problem it adds that solution to the database. RBR cannot learn by itself and requires a programmer to manually update the database.
4. Examples of real-world applications for both.
    - RBR: Network Firewall, Smart Home Automation
    - CBR: Medical Diagnosis Systems, Scenario-based Learning